



HSPS 班級點數銀行  
CLASS POINT BANK

SDD · 系統設計文件

# HSPS 點數銀行 系統設計文件

Software Design Document

|    |                   |
|----|-------------------|
| 版本 | v1.0 ( 草案 · 待審閱 ) |
|----|-------------------|

|    |                 |
|----|-----------------|
| 日期 | 2026 年 9 月 26 日 |
|----|-----------------|

|      |                               |
|------|-------------------------------|
| 對應需求 | HSPS 點數銀行 產品需求文件 ( PRD ) v1.0 |
|------|-------------------------------|

|     |                                                             |
|-----|-------------------------------------------------------------|
| 技術棧 | Vue 3 · Vite · Firebase ( Auth / Firestore ) · GitHub Pages |
|-----|-------------------------------------------------------------|

|    |                |
|----|----------------|
| 狀態 | 設計確認中 · 尚未進入開發 |
|----|----------------|



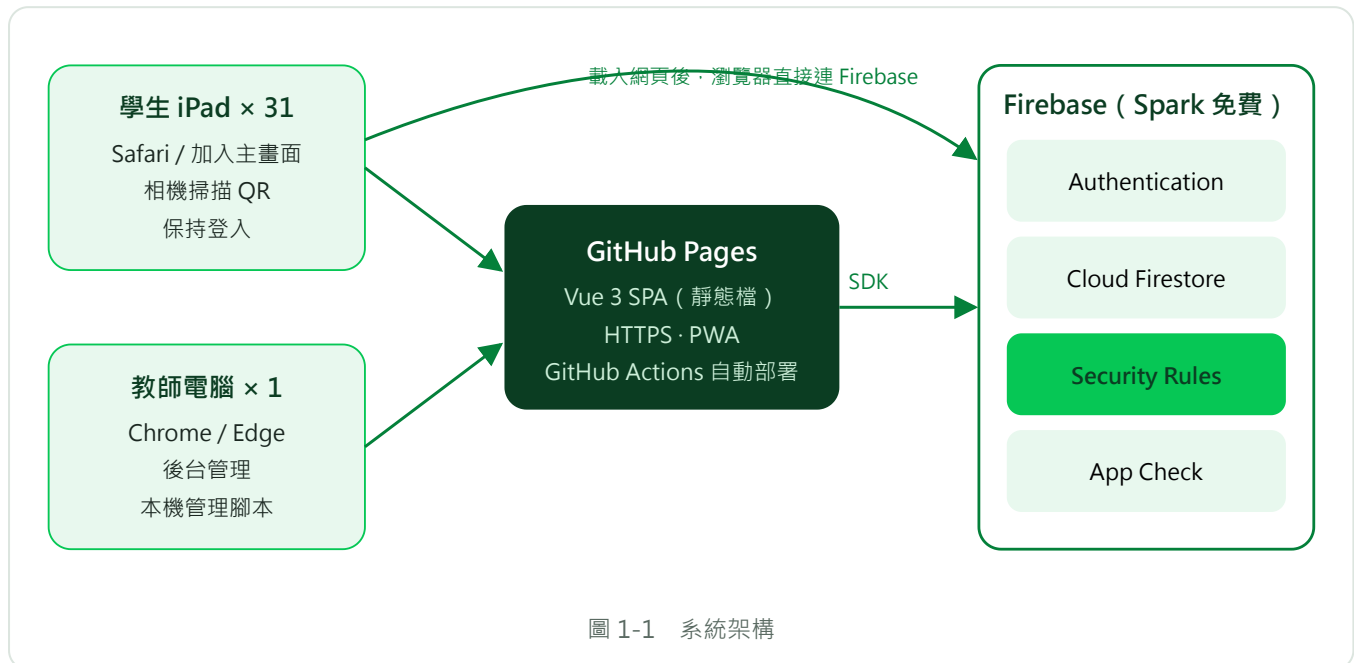
# 目錄

---

- 1 系統概觀與架構
- 2 技術選型
- 3 專案結構
- 4 資料模型
- 5 身分驗證與授權
- 6 核心流程設計
- 7 Firestore Security Rules
- 8 前端設計
- 9 QR 收付款設計
- 10 錯誤處理
- 11 部署與環境
- 12 安全設計
- 13 測試策略
- 14 容量與配額估算
- 15 維運、備份與管理腳本
- 16 風險與對策

# 1 系統概觀與架構

系統為純前端單頁應用 ( SPA )，靜態檔案部署於 GitHub Pages；資料與登入由 Firebase 提供。因為使用免費 Spark 方案、沒有自有後端，所有權限與資料完整性檢查都由 Firestore Security Rules 在資料庫端強制執行，前端檢查只負責使用者體驗。



## 1.1 設計原則

- **資料庫即防線**：前端程式可被竄改，因此每一條權限規則都寫在 Security Rules 並有自動化測試。
- **帳本為唯一真相**：`transactions` 只增不刪；`users.balance` 是帳本加總的快取，由原子交易維護，並提供對帳檢查。
- **學生只能「申請」**：學生唯一能建立的交易是「審核中的轉帳」；所有餘額變動都由教師端的原子交易完成。
- **即時同步**：以 Firestore `onSnapshot` 監聽，餘額與審核狀態自動更新。

## 2 技術選型

| 類別           | 選擇                                             | 理由                                          |
|--------------|------------------------------------------------|---------------------------------------------|
| 前端框架         | Vue 3 ( Composition API ) + TypeScript         | 見 2.1                                       |
| 建置工具         | Vite                                           | 啟動快、設定少，輸出靜態檔適合 GitHub Pages                |
| 路由           | Vue Router 4 ( Hash 模式 )                       | GitHub Pages 不支援 SPA 路由改寫；Hash 模式重新整理不會 404 |
| 狀態管理         | Pinia                                          | Vue 官方推薦，型別友善                               |
| 樣式           | Tailwind CSS 4                                 | 以設計權杖 ( 色彩、間距 ) 統一 logo 綠色系視覺               |
| Firebase SDK | firebase v11 ( 模組化 )                           | Auth、Firestore、App Check；支援 tree-shaking    |
| QR 產生        | qrcode                                         | 產生 SVG / Canvas QR 碼                        |
| QR 掃描        | qr-scanner ( nimiq )                           | 體積小、支援 iPad Safari 與 WebWorker 解碼           |
| CSV 解析       | PapaParse                                      | 處理 BOM、Big5 轉存的 UTF-8 CSV                   |
| Excel 匯出     | SheetJS ( 官方 CDN 套件 )                          | 產生 .xlsx                                    |
| 拖拉分組         | vuedraggable ( SortableJS )                    | 支援觸控與滑鼠                                     |
| 測試           | Vitest、@firebase/rules-unit-testing、Playwright | 單元、Rules、端對端                                |
| 本機模擬         | Firebase Emulator Suite                        | 開發與測試不動正式資料                                 |

### 2.1 為什麼選 Vue 而不是 React

- 單檔元件 ( .vue ) 直觀：HTML、邏輯、樣式在同一檔，日後老師或其他人接手維護門檻較低。
- 樣板程式少：表單雙向綁定 ( v-model ) 適合本系統大量的表單與勾選清單。
- 中文文件完整：Vue 官方提供繁體中文文件。
- 規模相符：本系統為中小型後台加行動頁面，兩者都能勝任；React 生態較大，但本案用不到。

## 3 專案結構

```
bank/  
├─ index.html  
├─ public/  
│  └─ logo.png  
# HSPS logo (原始檔)
```

```

├─ apple-touch-icon.png      # 180×180，iPad 加入主畫面
├─ icon-192.png / icon-512.png
├─ manifest.webmanifest
├─ src/
│  ├─ main.ts / App.vue
│  ├─ firebase.ts            # 初始化 App、Auth、Firestore、App Check
│  ├─ router/index.ts        # 路由與角色守衛
│  ├─ stores/                 # Pinia：auth、account、teacher、settings
│  ├─ domain/                # 純邏輯（可單元測試）
│  │  ├─ types.ts            # 資料型別
│  │  ├─ money.ts            # 金額驗證、格式化
│  │  ├─ csv.ts              # 學生名單解析與驗證
│  │  ├─ dates.ts            # Asia/Taipei 日期、本週區間
│  │  └─ codes.ts            # 收款碼產生
│  ├─ services/              # 與 Firestore 互動
│  │  ├─ users.ts transactions.ts approvals.ts adjustments.ts
│  │  ├─ groupAwards.ts reversals.ts paymentCodes.ts
│  │  └─ groups.ts presets.ts settings.ts accounts.ts export.ts
│  ├─ components/            # 共用元件：BalanceCard、TxList、TxRow、StudentPicker...
│  └─ views/
│     ├─ LoginView.vue
│     ├─ student/ HomeView HistoryView ReceiveView PayView ProfileView
│     └─ teacher/ DashboardView ApprovalsView StudentsView StudentDetailView
│                   AdjustView BatchView GroupAwardView LedgerView
│                   PresetsView GroupsView AccountsView SettingsView
├─ firestore.rules
├─ firestore.indexes.json
├─ firebase.json              # Emulator 設定
├─ scripts/
│  ├─ admin.mjs               # 本機管理腳本（重設密碼、初始化教師）
│  └─ service-account.json    # ⚠ 已列入 .gitignore，絕不提交
├─ tests/
│  ├─ unit/ rules/ integration/ e2e/
└─ .github/workflows/deploy.yml

```

## 4 資料模型 ( Cloud Firestore )

所有金額為整數。時間欄位使用 Firestore `Timestamp`，由 `serverTimestamp()` 或 `request.time` 產生；顯示時轉為 Asia/Taipei。

### 4.1 users/{uid}

文件 ID = Firebase Auth uid。一位教師 + 31 位學生。

| 欄位                                              | 型別                    | 說明               |
|-------------------------------------------------|-----------------------|------------------|
| <code>role</code>                               | 'student'   'teacher' | 角色，只能由教師或管理腳本設定  |
| <code>studentNo</code>                          | string                | 學號 ( 教師為空字串 )    |
| <code>name</code>                               | string                | 姓名               |
| <code>seatNo</code>                             | number                | 座號               |
| <code>groupId</code>                            | string   null         | 所屬組別             |
| <code>balance</code>                            | number ( int ≥ 0 )    | 目前餘額 ( 帳本加總的快照 ) |
| <code>active</code>                             | boolean               | false 時無任何存取權限   |
| <code>createdAt</code> / <code>updatedAt</code> | Timestamp             |                  |

### 4.2 transactions/{txId}：帳本

| 欄位                                          | 型別                                                             | 說明                                            |
|---------------------------------------------|----------------------------------------------------------------|-----------------------------------------------|
| <code>type</code>                           | 'reward'   'penalty'   'transfer'   'groupReward'   'reversal' | 交易類型                                          |
| <code>amount</code>                         | int > 0                                                        | 一律正數；方向由 from / to 決定                         |
| <code>fromUid</code> / <code>toUid</code>   | string   null                                                  | null 代表「銀行」 ( 加點時 from 為 null，扣點時 to 為 null ) |
| <code>fromName</code> / <code>toName</code> | string   null                                                  | 姓名快照                                          |
| <code>participants</code>                   | string[]                                                       | 涉及的學生 uid，供查詢與權限使用                            |

|                       |                                           |                                                              |
|-----------------------|-------------------------------------------|--------------------------------------------------------------|
| status                | 'pending'   'approved'   'rejected'       | 教師直接加扣點建立時即為 approved                                        |
| title                 | string                                    | 摘要，例如「表現優良」「小組冠軍（第3組）」                                       |
| note                  | string ≤ 50                               | 原因或轉帳目的                                                      |
| category              | 'buy'   'repay'   'gift'   'other'   null | 轉帳分類：買東西、還錢、贈送、其他                                            |
| presetId              | string   null                             | 來源預設事件                                                       |
| batchId               | string   null                             | 批次編號（批次加扣點、小組獎勵）                                             |
| group                 | map   null                                | { id, name, rank: 1 2, tied: bool, awardDate: 'YYYY-MM-DD' } |
| codeId                | string   null                             | 轉帳使用的收款碼                                                     |
| balanceAfter          | map<uid, int>                             | 生效當下各方餘額快照                                                   |
| reversalOf            | string   null                             | 本筆為沖正時，指向原交易                                                 |
| reversedBy            | string   null                             | 原交易被沖正後，指向沖正交易                                               |
| rejectReason          | string   null                             | 駁回原因                                                         |
| createdBy / createdAt | uid / Timestamp                           | 建立者與時間                                                       |
| decidedBy / decidedAt | uid / Timestamp                           | 核准或駁回者與時間                                                    |

### 4.3 paymentCodes/{codeId}：一次性收款碼

| 欄位                        | 型別                  | 說明                                                 |
|---------------------------|---------------------|----------------------------------------------------|
| 文件 ID                     | 8 碼                 | 字元集 A-Z、2-9 去除易混淆的 I、O、0、1 ( 32 種字元，約 40 bits 隨機 ) |
| toUid / toName / toSeatNo |                     | 收款人 ( 快照，付款方不必讀取他人帳戶 )                             |
| createdAt / expiresAt     | Timestamp           | 有效期限預設 3 分鐘，最長 10 分鐘                               |
| used / usedBy / txId      | bool / uid / string | 使用狀態，與轉帳申請於同一批次寫入                                  |

## 4.4 其他集合

| 集合                    | 欄位                                                                                                                                                           |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| presets/{id}          | name, kind('add' 'deduct'), amount, color, sortOrder, active                                                                                                 |
| groups/{id}           | name, color, sortOrder, active                                                                                                                               |
| groupAwards/{batchId} | awardDate, champions[{id,name}], runnersUp[{id,name}], championAmount, runnerUpAmount, recipients[uid], excluded[uid], total, reversed, createdBy, createdAt |
| settings/app          | codeTtlMinutes(1-10), championAmount, runnerUpAmount, className                                                                                              |

## 4.5 索引 ( firestore.indexes.json )

| 集合           | 欄位                                             | 用途        |
|--------------|------------------------------------------------|-----------|
| transactions | participants (array-contains) + createdAt desc | 學生明細      |
| transactions | status + createdAt asc                         | 待審核清單     |
| transactions | batchId + createdAt                            | 批次查詢與整批沖正 |
| transactions | type + createdAt desc                          | 交易紀錄篩選    |
| groupAwards  | awardDate + reversed                           | 同日重複發放檢查  |

其他篩選 ( 學生、組別、分類 ) 以日期區間查詢後在前端過濾；本班資料量 ( 約 2 萬筆 / 學期 ) 足以負荷。

## 5 身分驗證與授權

### 5.1 帳號格式

- 使用 Firebase Auth 「電子郵件 / 密碼」登入。學生不需要真實 email，登入畫面輸入學號，程式轉為 `{學號}@student.hspsbank.local`。
  - 此網域不收信，僅作為識別；實作時先於 Emulator 驗證格式可被接受。
- 教師使用自己的真實 email，可使用 Firebase 內建的「忘記密碼」信件。
- 登入狀態使用 `browserLocalPersistence`（一人一台 iPad，保持登入）。

### 5.2 角色判斷

- 角色存於 `users/{uid}.role`，Security Rules 以 `get()` 讀取判斷；前端路由守衛只作為體驗輔助。
- 名冊制：Auth 帳號必須在 `users` 有對應文件且 `active == true`，才有任何讀寫權限。因此即使有人用公開的 API 金鑰自行註冊 Auth 帳號，也無法存取任何資料。

### 5.3 帳號生命週期

| 動作     | 做法                                                                                                                                                                   |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 初始化教師  | 於 Firebase Console 建立教師 Auth 帳號，再執行 <code>node scripts/admin.mjs init-teacher &lt;email&gt;</code> 寫入 <code>users/{uid}</code> （ <code>role = teacher</code> ）。只做一次。 |
| 建立學生   | 後台 CSV 匯入：以第二個 Firebase App 實例呼叫 <code>createUserWithEmailAndPassword</code> ，教師本身的登入不受影響；成功後由教師身分寫入 <code>users/{newUid}</code> 。                                   |
| 學生改密碼  | 學生於個人頁輸入舊密碼重新驗證後，呼叫 <code>updatePassword</code> 。                                                                                                                    |
| 重設學生密碼 | 免費方案下瀏覽器無法修改他人密碼，因此由教師在電腦執行 <code>node scripts/admin.mjs reset-password &lt;學號&gt; &lt;新密碼&gt;</code> （Admin SDK）。                                                   |
| 停用     | 後台將 <code>active</code> 設為 <code>false</code> ；Rules 立即拒絕其所有存取。                                                                                                      |

## 6 核心流程設計

所有會改變餘額的操作都使用 Firestore `runTransaction` (讀取後寫入，衝突時自動重試)，確保多人同時操作仍正確。

### 6.1 學生送出轉帳申請

以 `writeBatch` 同時寫入兩份文件，Rules 用 `getAfter()` 確認兩者一致，收款碼因此只能被使用一次。

```
const txRef = doc(collection(db, 'transactions')); // 預先取得 ID
const batch = writeBatch(db);
batch.set(txRef, {
  type: 'transfer', status: 'pending', amount,
  fromUid: me.uid, fromName: me.name, toUid: code.toUid, toName: code.toName,
  participants: [me.uid, code.toUid],
  title: `轉帳給 ${code.toName}`, note, category, codeId,
  createdBy: me.uid, createdAt: serverTimestamp(), ...nullFields,
});
batch.update(doc(db, 'paymentCodes', codeId), { used: true, usedBy: me.uid, txId: txRef.id });
await batch.commit(); // 任一條件不符 → 整批失敗
```

### 6.2 教師核准轉帳

```
await runTransaction(db, async (t) => {
  const tx = await t.get(txRef);
  if (tx.data().status !== 'pending') throw new BankError('ALREADY_DECIDED');
  const from = await t.get(userRef(tx.data().fromUid));
  const to = await t.get(userRef(tx.data().toUid));
  const amt = tx.data().amount;
  if (from.data().balance < amt) throw new BankError('INSUFFICIENT_BALANCE');
  const fb = from.data().balance - amt, tb = to.data().balance + amt;
  t.update(from.ref, { balance: fb, updatedAt: serverTimestamp() });
  t.update(to.ref, { balance: tb, updatedAt: serverTimestamp() });
  t.update(txRef, { status: 'approved', decidedBy: teacherUid, decidedAt: serverTimestamp(),
    balanceAfter: { [from.id]: fb, [to.id]: tb } });
});
```

- 駁回：只更新 `status = 'rejected'`、`rejectReason`、`decidedBy` / `decidedAt`，不動餘額。
- 核准所選：依送出時間先後逐筆執行上述交易，逐筆回報結果（成功或餘額不足），不會因一筆失敗而中斷。

### 6.3 個別 / 批次加扣點、小組獎勵

三者共用同一個函式 `applyAdjustments(items, meta)`，在單一 `runTransaction` 內完成：

1. 讀取所有目標學生的 `users` 文件（全班最多 31 筆）。
2. 扣點時檢查每位學生餘額，不足者列入錯誤清單並中止（批次扣點畫面會先讓老師排除不足者）。
3. 為每位學生寫入 `users.balance` 與一筆 `transactions`（`status = 'approved'`，共用 `batchId`）。
4. 小組獎勵另寫一筆 `groupAwards/{batchId}`。

單次交易最多 500 個寫入；全班  $31 \times 2 + 1 = 63$  個寫入，遠低於上限。

### 小組獎勵：並列名次

`champions` 與 `runnersUp` 皆為陣列，UI 以複選實作。當某名次選了 2 組以上時，交易的 `group.tied = true`，學生明細顯示「並列冠軍」。送出前查詢 `groupAwards` 中 `awardDate == 當日` 且 `reversed == false`，若已有紀錄則跳出確認對話框。

## 6.4 沖正

```
await runTransaction(db, async (t) => {
  const orig = await t.get(origRef);
  const o = orig.data();
  if (o.status !== 'approved') throw new BankError('NOT_APPROVED');
  if (o.reversedBy) throw new BankError('ALREADY_REVERSED');
  if (o.type === 'reversal') throw new BankError('CANNOT_REVERSE_REVERSAL');
  // 反向：原本的 to 變成 from
  const revFrom = o.toUid, revTo = o.fromUid;
  if (revFrom) { const u = await t.get(userRef(revFrom));
    if (u.data().balance < o.amount) throw new BankError('NEGATIVE_AFTER_REVERSAL'); }
  // ...更新雙方 balance、建立 type='reversal' 交易 (reversalOf = 原 ID)、原交易寫入 reversedBy
});
```

整批沖正：以 `batchId` 查出所有未沖正交易，在同一個交易中全部沖正；小組獎勵同時將 `groupAwards.reversed` 設為 `true`。

## 7 Firestore Security Rules ( 草案 )

以下為核心規則草案，實作時會以 7.2 的測試矩陣逐條驗證。

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {

    function signedIn() { return request.auth != null; }
    function userDoc(uid) { return get(/databases/{db}/documents/users/{uid}).data; }
    function me() { return userDoc(request.auth.uid); }
    function isActive() { return signedIn()
      && exists(/databases/{db}/documents/users/{request.auth.uid})
      && me().active == true; }
    function isTeacher() { return isActive() && me().role == 'teacher'; }
    function isStudent() { return isActive() && me().role == 'student'; }

    // ----- 帳戶 -----
    match /users/{uid} {
      allow read: if isTeacher() || (isStudent() && request.auth.uid == uid);
      allow create, update: if isTeacher();
      allow delete: if false; // 只能停用，不能刪除
    }

    // ----- 設定類：學生唯讀 -----
    match /groups/{id} { allow read: if isActive(); allow write: if isTeacher(); }
    match /presets/{id} { allow read: if isTeacher(); allow write: if isTeacher(); }
    match /settings/{id} { allow read: if isActive(); allow write: if isTeacher(); }
    match /groupAwards/{id} { allow read, write: if isTeacher(); }

    // ----- 一次性收款碼 -----
    match /paymentCodes/{codeId} {
      allow get: if isStudent();
      allow list: if false; // 禁止列舉
      allow create: if isStudent()
        && codeId.matches('^A-HJ-NP-Z2-9]{8}$')
        &&
        request.resource.data.keys().hasOnly(['toUid', 'toName', 'toSeatNo', 'createdAt', 'expiresAt', 'used'])
        && request.resource.data.toUid == request.auth.uid
        && request.resource.data.toName == me().name
        && request.resource.data.toSeatNo == me().seatNo
        && request.resource.data.createdAt == request.time
        && request.resource.data.expiresAt <= request.time + duration.value(10, 'm')
        && request.resource.data.used == false;
      allow update: if isStudent()
        && resource.data.used == false
        && request.time < resource.data.expiresAt
        && resource.data.toUid != request.auth.uid
        && request.resource.data.diff(resource.data).affectedKeys().hasOnly(['used', 'usedBy', 'txId'])
        && request.resource.data.used == true
        && request.resource.data.usedBy == request.auth.uid
        && getAfter(/databases/{db}/documents/transactions/{request.resource.data.txId})
          .data.codeId == codeId;
      allow delete: if isStudent() && resource.data.toUid == request.auth.uid
        && resource.data.used == false;
    }
  }
}
```

```
// ----- 帳本 -----
match /transactions/{txId} {
  allow read:  if isTeacher() || (isStudent() && request.auth.uid in resource.data.participants);
  allow create: if isTeacher() || validTransferRequest(txId);
  allow update: if isTeacher() && validTeacherUpdate();
  allow delete: if false; // 帳本只增不刪
}

function validTransferRequest(txId) {
  let d      = request.resource.data;
  let before = get(/databases/${db}/documents/paymentCodes/${d.codeId}).data;
  let after  = getAfter(/databases/${db}/documents/paymentCodes/${d.codeId}).data;
  return isStudent()
    && d.type == 'transfer' && d.status == 'pending'
    && d.fromUid == request.auth.uid && d.fromName == me().name
    && d.amount is int && d.amount > 0 && d.amount <= me().balance
    && d.category in ['buy','repay','gift','other']
    && d.note is string && d.note.size() > 0 && d.note.size() <= 50
    && before.used == false && request.time < before.expiresAt
    && d.toUid == before.toUid && d.toName == before.toName
    && d.toUid != request.auth.uid
    && userDoc(d.toUid).active == true
    && d.participants == [d.fromUid, d.toUid]
    && d.createdBy == request.auth.uid && d.createdAt == request.time
    && d.balanceAfter == {} && d.reversalOf == null && d.reversedBy == null
    && after.used == true && after.usedBy == request.auth.uid && after.txId == txId;
}

function validTeacherUpdate() {
  let changed = request.resource.data.diff(resource.data).affectedKeys();
  return !changed.hasAny(['type','amount','fromUid','toUid','participants',
    'createdBy','createdAt','reversalOf','batchId','codeId'])
    // 狀態只能 pending → approved / rejected
    && (resource.data.status == 'pending' || !changed.hasAny(['status']))
    // reversedBy 只能由 null 設定一次
    && (resource.data.reversedBy == null || !changed.hasAny(['reversedBy']));
}
}
}
```

### 已知限制

Rules 無法跨文件驗證「教師寫入的餘額是否等於帳本加總」。教師端以 `runTransaction` 保證一致，並提供對帳檢查 ( F-T14 ) 偵測異常。教師帳號是唯一高權限帳號，須設定強密碼。

## 7.2 Rules 測試矩陣 ( 節錄 )

| 情境                          | 預期 |
|-----------------------------|----|
| 未登入讀取任何集合                   | 拒絕 |
| 已登入但不在名冊 ( 自行註冊的帳號 ) 讀取任何集合 | 拒絕 |
| 停用學生讀取自己的帳戶                 | 拒絕 |

|                                          |    |
|------------------------------------------|----|
| 學生讀取他人 users 文件、或查詢不含自己的交易               | 拒絕 |
| 學生修改自己的 balance / role / groupId         | 拒絕 |
| 學生建立 reward / groupReward / reversal 交易  | 拒絕 |
| 學生建立 status = approved 的轉帳               | 拒絕 |
| fromUid 不是自己；金額 0、負數、小數、字串；超過餘額          | 拒絕 |
| 收款碼已用、過期、不存在、收款人不一致、付款給自己                | 拒絕 |
| 只建立轉帳而未同批標記收款碼（或反之）                      | 拒絕 |
| 列舉 paymentCodes 集合                       | 拒絕 |
| 任何人刪除交易或帳戶                               | 拒絕 |
| 教師修改已核准交易的 amount、或把 approved 改回 pending | 拒絕 |
| 學生以合法收款碼送出轉帳申請                           | 允許 |
| 教師核准、駁回、加扣點、沖正                           | 允許 |

## 8 前端設計

### 8.1 路由

| 路徑 ( Hash )                                                                                                         | 角色 | 頁面             |
|---------------------------------------------------------------------------------------------------------------------|----|----------------|
| <code>#/login</code>                                                                                                | 公開 | 登入             |
| <code>#/</code>                                                                                                     | 學生 | 首頁             |
| <code>#/history</code>                                                                                              | 學生 | 交易明細           |
| <code>#/receive</code>                                                                                              | 學生 | 收款 ( 顯示 QR )   |
| <code>#/pay?c=CODE</code>                                                                                           | 學生 | 付款 ( 掃描或帶入代碼 ) |
| <code>#/me</code>                                                                                                   | 學生 | 個人資料、修改密碼      |
| <code>#/t</code>                                                                                                    | 教師 | 儀表板            |
| <code>#/t/approvals</code>                                                                                          | 教師 | 轉帳審核           |
| <code>#/t/students</code> 、<br><code>#/t/students/:uid</code>                                                       | 教師 | 學生總覽、個人頁       |
| <code>#/t/adjust</code> 、 <code>#/t/batch</code>                                                                    | 教師 | 個別加扣點、批次加扣點    |
| <code>#/t/group-award</code>                                                                                        | 教師 | 小組獎勵與歷史        |
| <code>#/t/ledger</code>                                                                                             | 教師 | 交易紀錄、沖正、匯出     |
| <code>#/t/presets</code> 、<br><code>#/t/groups</code> 、<br><code>#/t/accounts</code> 、<br><code>#/t/settings</code> | 教師 | 預設事件、分組、帳號、設定  |

路由守衛：未登入導向 `#/login`；學生進入 `#/t/*` 導回 `#/`；教師登入後預設進入 `#/t`。

### 8.2 即時資料訂閱

| 訂閱    | 查詢                                                                                 | 使用頁面  |
|-------|------------------------------------------------------------------------------------|-------|
| 我的帳戶  | <code>doc(users/{uid})</code>                                                      | 學生全頁面 |
| 我的交易  | <code>participants array-contains uid, orderBy<br/>createdAt desc, limit 30</code> | 首頁、明細 |
| 收款碼狀態 | <code>doc(paymentCodes/{codeId})</code>                                            | 收款頁   |

|      |                                                     |              |
|------|-----------------------------------------------------|--------------|
| 待審核  | <code>status == 'pending', orderBy createdAt</code> | 教師全頁面 ( 徽章 ) |
| 全班帳戶 | <code>users where role == 'student'</code>          | 教師後台         |

## 8.3 視覺實作

- Tailwind 設計權杖依 PRD 第 10 章 :
  - `brand #06C755` 、 `brand-dark #04803A` 、 `ink #0B3D22` 、 `brand-soft #E8F8EE`
  - 支出 `#B42318` 、 審核中 `#B45309`
- logo 應用 :
  - 登入頁 120px 、 頁首 36px
  - favicon
  - `apple-touch-icon` 180×180 · 以原始 logo 縮放產生
  - `manifest.webmanifest` : `theme_color #06C755` 、 `display: standalone`
- 字型 : 系統字型堆疊 `-apple-system, "PingFang TC", "Microsoft JhengHei", sans-serif` ; 金額使用 `font-variant-numeric: tabular-nums` 。
- 金額格式化集中於 `domain/money.ts` : `Intl.NumberFormat('zh-TW')` 。

## 9 QR 收付款設計

### 9.1 收款碼產生

```
const ALPHABET = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789'; // 32 字，去除 I 0 O 1
function newCodeId() {
  const bytes = crypto.getRandomValues(new Uint8Array(8));
  return Array.from(bytes, b => ALPHABET[b & 31]).join('');
}
// 建立：setDoc(paymentCodes/{id}, {..., createdAt: serverTimestamp(),
// expiresAt: Timestamp.fromMillis(Date.now() + ttl)})
// ID 碰撞時 create 會被 Rules 視為 update 而拒絕 → 換一個 ID 重試
```

- QR 內容為網址：`https://<帳號>.github.io/<repo>/#/pay?c=ABCD2345`。用 App 內掃描器或 iPad 內建相機掃描都能直接開到付款頁。
- `expiresAt` 由用戶端計算，但 Rules 限制不得超過伺服器時間加 10 分鐘；有效性一律以伺服器時間判斷。
- 收款頁監聽該文件：`used` 變為 `true` 時，讀取 `txId` 對應的交易，顯示付款人與金額。
- 離開頁面（`onUnmounted`、`visibilitychange`）時，刪除未使用的收款碼。

### 9.2 掃描

- 使用 `qr-scanner`，指定後鏡頭 `preferredCamera: 'environment'`。
- 必須在 HTTPS 下執行（GitHub Pages 預設為 HTTPS）；iPadOS「加入主畫面」模式亦支援相機。
- 掃描結果只接受本站網址或 8 碼代碼格式，其他內容一律忽略，防止被導向外部網站。
- 取得代碼後 `getDoc(paymentCodes/{c})`：
  - 不存在、已使用或已過期 → 顯示錯誤。
  - 有效 → 顯示收款人姓名與座號，進入填寫步驟。

## 10 錯誤處理

| 錯誤碼                     | 使用者看到的訊息            |
|-------------------------|---------------------|
| INSUFFICIENT_BALANCE    | 餘額不足（目前 ○○ 點）       |
| ALREADY_DECIDED         | 這筆申請已經處理過了          |
| CODE_INVALID            | 此收款碼已使用或已失效，請對方重新產生 |
| ALREADY_REVERSED        | 這筆交易已經沖正過了          |
| NEGATIVE_AFTER_REVERSAL | ○○○ 的餘額不足，無法沖正      |

|                                      |                    |
|--------------------------------------|--------------------|
| <code>permission-denied</code>       | 沒有權限執行此操作（並記錄於主控台） |
| <code>unavailable</code> （離線）        | 網路連線中斷，請稍後再試       |
| <code>auth/invalid-credential</code> | 學號或密碼錯誤            |
| <code>auth/too-many-requests</code>  | 嘗試次數過多，請稍後再試       |

Firestore 離線寫入快取關閉（`memoryLocalCache`）：交易類操作必須即時確認成功，不允許離線排隊。

## 11 部署與環境

| 環境   | 說明                                                                                   |
|------|--------------------------------------------------------------------------------------|
| 本機開發 | <code>npm run dev</code> 搭配 Firebase Emulator ( Auth 、 Firestore ) ，測試資料由 seed 腳本產生。 |
| 正式環境 | GitHub Pages 加上 Firebase 正式專案。                                                       |

### 11.1 CI/CD ( GitHub Actions )

```
on: push (main)
jobs:
  test:
    npm ci → lint → typecheck → vitest(unit)
    → firebase emulators:exec "vitest rules + integration"
    → playwright (webkit iPad 、 chromium desktop)
  deploy: needs test → vite build (base='/<repo>/' ) → actions/deploy-pages
```

- Firebase 前端設定 ( apiKey 等 ) 存於 repo Variables ，建置時注入 `VITE_FIREBASE_*` 。此設定本來就公開於前端，安全性由 Rules 與 App Check 保障。
- Security Rules 與索引由教師電腦手動部署： `firebase deploy --only firestore` ，部署前須先通過 Rules 測試。
- Firebase Auth 「授權網域」須加入 `<帳號>.github.io` 。

## 12 安全設計

| 威脅             | 對策                                                                                              |
|----------------|-------------------------------------------------------------------------------------------------|
| 學生竊改前端直接改餘額    | Rules : <code>users</code> 只有教師可寫；學生只能建立 pending 轉帳。                                            |
| 重複使用 QR 碼、截圖轉傳 | 一次性收款碼 + 伺服器時間過期 + 與轉帳同批寫入 ( <code>getAfter</code> ) 。                                          |
| 猜測收款碼          | 40 bits 隨機、禁止列舉、3 分鐘失效。                                                                         |
| 重複核准、同時核准      | <code>runTransaction</code> 檢查 <code>status == pending</code> ，衝突時自動重試。                         |
| 偷看他人資料         | Rules 以 <code>participants</code> 與 <code>uid</code> 限制讀取；收款人資訊以快照存於收款碼，不需讀取他人帳戶。               |
| 自行註冊帳號         | 名冊制：不在 <code>users</code> 或未啟用者無任何權限。                                                           |
| 濫用 API 金鑰、機器人  | Firebase App Check ( reCAPTCHA v3 ) 強制驗證 Firestore 請求；於 Google Cloud 限制 API 金鑰的 HTTP referrer 。 |
| XSS            | Vue 預設跳脫輸出、禁用 <code>v-html</code> ；目的與原因欄位限制 50 字；CSP meta 限制資源來源。                              |
| 暴力破解密碼         | Firebase Auth 內建節流；密碼至少 6 碼；登入錯誤訊息不透露細節。                                                        |
| 管理金鑰外洩         | <code>service-account.json</code> 列入 <code>.gitignore</code> ，只存在教師電腦；CI 不使用。                   |
| 相依套件漏洞         | Dependabot、 <code>npm audit</code> 。                                                            |
| 教師帳號被盜         | 強密碼；建議教師帳號於 Firebase 啟用多重驗證 ( 需評估方案支援度 )；定期匯出備份。                                                |

### 12.1 CSP ( 初稿 )

```
default-src 'self';
script-src 'self' https://www.google.com https://www.gstatic.com;
connect-src 'self' https://*.googleapis.com https://*.firebaseio.com https://www.google.com;
frame-src https://www.google.com https://*.firebaseapp.com;
img-src 'self' data: blob:; style-src 'self' 'unsafe-inline'; worker-src 'self' blob;;
```

## 13 測試策略

| 層級    | 工具                                         | 範圍                                                                                                                                                                               |
|-------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 單元    | Vitest                                     | <ul style="list-style-type: none"><li><code>domain/*</code>：金額驗證、CSV 解析（重複學號、缺欄、BOM、全形）、日期與本週區間、收款碼格式。</li><li>元件：付款表單驗證、倒數失效。</li></ul>                                         |
| Rules | @firebase/rules-unit-testing + Emulator    | 7.2 測試矩陣全部項目，每條規則都有允許與拒絕案例。                                                                                                                                                      |
| 整合    | Vitest + Emulator                          | <ul style="list-style-type: none"><li>核准、駁回，以及並行核准同一筆（只成功一次）。</li><li>超額申請逐筆核准。</li><li>批次加扣點、小組獎勵（並列、請假、重複日期）。</li><li>單筆與整批沖正、沖正後負餘額阻擋。</li><li>收款碼生命週期；對帳：帳本加總等於餘額。</li></ul> |
| 端對端   | Playwright ( WebKit iPad 設定檔、Chromium 桌面 ) | <ul style="list-style-type: none"><li>教師匯入學生 → 批次加點 → 學生 A 收款、學生 B 付款（以注入代碼取代相機）→ A 畫面即時更新 → 教師核准 → 餘額與明細驗證 → 沖正還原。</li><li>學生開啟教師路由被導回。</li></ul>                               |
| 安全    | 手動滲透檢查                                     | 以學生帳號在瀏覽器主控台直接呼叫 SDK，嘗試 7.2 的所有攻擊；檢查 CSP 與 XSS。                                                                                                                                  |
| 實機驗收  | 真實 iPad                                    | 相機權限、加入主畫面、直橫向、校園 Wi-Fi 下的載入速度。                                                                                                                                                  |

合併到 main 前，全部自動化測試必須通過；Rules 測試覆蓋 `firestore.rules` 中每一個 `allow` 敘述。

## 14 容量與配額估算 ( Spark 免費方案 )

| 項目                                          | 每日估算         | 免費額度       | 占比  |
|---------------------------------------------|--------------|------------|-----|
| 文件讀取：31 人 × 每日 4 次開啟 × 約 40 筆，加上教師後台約 3,000 | 約 8,000      | 50,000     | 16% |
| 文件寫入：加扣點、小組獎勵、轉帳約 150 筆 × 2~3 寫入            | 約 500        | 20,000     | 3%  |
| 刪除（收款碼）                                     | 約 50         | 20,000     | <1% |
| 儲存：約 2 萬筆交易 / 學期 × 1 KB                     | 約 20 MB / 學期 | 1 GB       | 2%  |
| Auth 使用者                                    | 32           | 不限（電子郵件登入） | —   |

GitHub Pages：每月 100 GB 流量、網站 1 GB，本系統打包後約 1 MB，遠低於上限。

## 15 維運、備份與管理腳本

### 15.1 管理腳本 scripts/admin.mjs (教師電腦執行)

| 指令                                                 | 用途                     |
|----------------------------------------------------|------------------------|
| <code>init-teacher &lt;email&gt;</code>            | 將既有 Auth 帳號設為教師 (首次建置) |
| <code>reset-password &lt;學號&gt; &lt;新密碼&gt;</code> | 重設學生密碼                 |
| <code>backup [輸出資料夾]</code>                        | 匯出全部集合為 JSON           |
| <code>reconcile</code>                             | 對帳：比對帳本加總與餘額           |

### 15.2 備份策略

- Spark 方案不支援排程匯出，因此提供兩種手動備份：後台「全資料備份」按鈕 (Excel / JSON) 與管理腳本 `backup`。
- 建議每月初與學期末各備份一次，存於學校雲端硬碟。

### 15.3 監控

- Firebase Console 用量頁面；接近額度 80% 時檢查是否有異常讀取。
- 後台對帳檢查 (F-T14) 每週執行一次。

## 16 風險與對策

| 風險                     | 影響 | 對策                                               |
|------------------------|----|--------------------------------------------------|
| Security Rules 寫錯導致漏洞  | 高  | 完整 Rules 測試矩陣、CI 強制通過、上線前手動滲透檢查                  |
| 教師端錯誤造成餘額與帳本不一致        | 中  | 所有餘額變動走 <code>runTransaction</code> ；對帳檢查；沖正機制修正 |
| 免費方案無法在網頁上重設密碼         | 低  | 本機管理腳本；未來可升級 Blaze 改用 Cloud Functions            |
| iPad 相機權限被拒或掃描失敗       | 低  | 8 碼代碼手動輸入備援；權限說明頁                                |
| 校園網路不穩                 | 中  | 明確的離線提示；交易操作不允許離線排隊，避免重複送出                       |
| Firebase 或 GitHub 服務中斷 | 低  | 定期備份；中斷期間暫以紙本記錄，恢復後由教師補登                         |
| 學生共用或外流密碼              | 中  | 轉帳需教師審核；可重設密碼、停用帳號；明細可追溯                         |